Co-Work 2027 - Rutix: diseno integral

Nov 7, 2026 . @ezequiel

Rutix es una app para compartir gastos de viajes en auto: un conductor publica un viaje y los pasajeros reservan asientos, dividiendo el costo. Este documento registra el diseno integral de la Unidad 4 (Co-Work 2027), tomando como base el proyecto real que ya esta en desarrollo. El registro de prompts y decisiones vive en `Expediente de co-work`.

Dia 1 - Problema, usuarios y alcance

Necesidad. Dividir el costo de un viaje en auto entre quien lo hace de todos modos (conductor) y quienes necesitan el mismo trayecto (pasajeros): la app gestiona la publicacion del viaje, la reserva de asientos y el cobro de la parte correspondiente a cada pasajero.

Usuarios

- Conductor: publica viajes (origen, destino, fecha, horarios, asientos disponibles, valor por asiento, si pide sena), administra su perfil, sus vehiculos y sus medios de cobro (cuenta bancaria, tarjeta), confirma o rechaza reservas.
- Pasajero: busca viajes por ciudad de partida/destino y fecha, reserva uno o mas asientos, paga por efectivo, transferencia o Mercado Pago, puede cancelar una reserva pendiente.
- Un mismo perfil puede ser conductor en un viaje y pasajero en otro: no son roles separados, sino un unico perfil (PerfilRutix) vinculado a una cuenta de usuario.

Contexto tecnico de partida (relevante para el alcance, porque ya hay una base construida)

- Frontend: app movil hibrida (Ionic + Angular, Capacitor) con login/JWT, busqueda y detalle de viajes, reserva, perfil, vehiculos y billetera.
- Backend: microservicio Spring Boot (Java 21) con JPA/MariaDB, pensado para integrarse a un ecosistema de microservicios propio (Eureka, Config Server, API Gateway, Spring Boot Admin, RabbitMQ, Zipkin, Prometheus/Grafana).
- Dominios ya modelados: PerfilRutix (perfil, vehiculos, tarjetas, cuentas bancarias) y Viajes (viaje, reserva, pago), con estados definidos (viaje: creado/en curso/finalizado/cancelado; reserva: pendiente/confirmada/pagada/rechazada/cancelada; pago: pendiente/confirmado/error) y un patron Strategy para medios de cobro externos (efectivo y Mercado Pago, hoy sin la logica real implementada).

- Valoracion: cada perfil tiene una valoracion y cada reserva registra un puntaje; el sistema de calificacion post-viaje ya esta contemplado en el modelo, aunque falta definir como se calcula.

Alcance de este trabajo

- Se disena el dominio de Viajes + Perfil: publicar/buscar viajes, reservar, confirmar/rechazar/cancelar, cobrar, calificar.
- Se disena la verificacion formal de identidad a nivel de perfil, en dos niveles: DNI validado habilita actuar como pasajero; DNI + licencia de conducir validados habilitan actuar como conductor de un viaje.
- No se disena en esta entrega: matching automatico de viajes por IA, chat en tiempo real conductor-pasajero, facturacion fiscal, ni la infraestructura de observabilidad (Prometheus/Grafana/Zipkin), que es transversal a todo el ecosistema y no especifica de Rutix.
- Se asume que el backend sigue el estilo arquitectonico ya elegido (microservicios con Eureka + Gateway) y el patron de Comando/Consulta que ya usa el codigo existente.

Supuestos iniciales a validar

- El valor del viaje es por asiento y lo fija el conductor; no hay tarifa dinamica.
- La unidad de cobro es la reserva, no el viaje completo: un viaje puede tener pagos en distintos estados segun el pasajero.
- La verificacion de identidad es un dato del PerfilRutix, en dos niveles: un perfil puede reservar como pasajero solo si tiene el DNI validado; puede publicar un viaje como conductor solo si tiene DNI y licencia de conducir validados (el DNI validado es, por lo tanto, condicion tanto para pasajero como para conductor, y la licencia se suma solo para conductor). Se resuelve con revision humana; diseno extensible a revision automatica.

> Decision registrada: verificacion de identidad en dos niveles sobre el PerfilRutix. DNI validado habilita el rol de pasajero. DNI + licencia de conducir validados habilitan el rol de conductor en un viaje. Revision humana en esta etapa, con diseno extensible a revision automatica. No quedan pendientes abiertos en el alcance del Dia 1.

Dia 2 - Alternativas evaluadas

La pregunta central del diseno: como se conectan conductor y pasajero para compartir un viaje. Se evaluaron dos enfoques.

	Enfoque A: viaje programado con reserva anticipada	Enfoque B: matching on-demand en tiempo real
Como funciona	El conductor publica un viaje con fecha, hora y asientos fijos. Los pasajeros lo buscan y reservan con anticipacion.	El pasajero pide un viaje ahora; el sistema busca conductores disponibles en ese momento y los empareja automaticamente.
Beneficios	Simple de implementar y de entender; el pasajero sabe con certeza cuando y con quien viaja; se presta a trayectos de media/larga distancia planificados.	Util para traslados cortos e inmediatos; mayor dinamismo.
Limites	No sirve para necesidades inmediatas ("necesito salir ya").	Exige ubicacion en tiempo real, matching geografico y conductores activos simultaneamente; mucho mas complejo de operar y de garantizar calidad de servicio.
Dependencias	Busqueda por filtros (ciudad, fecha); no depende de geolocalizacion en vivo.	Geolocalizacion continua, un motor de matching, y masa critica de conductores conectados en el momento.
Riesgos	Baja oferta si pocos conductores publican en una fecha/ruta puntual.	Sin suficiente densidad de usuarios el matching falla o tarda; mayor superficie de fallos (ubicacion, concurrencia, timeouts).

Decision inicial: se elige el Enfoque A (viaje programado con reserva anticipada). Es el que ya esta modelado en el codigo existente (Viaje con fecha/hora fija, asientos y reservas), responde mejor al caso de uso tipico de Rutix (compartir gastos en un trayecto ya planeado, por ejemplo un viaje de fin de semana o un commute regular) y no exige la infraestructura de tiempo real que el Enfoque B necesitaria para funcionar bien. El Enfoque B queda registrado como una extension posible a futuro, no como parte de este diseno.

> Pendiente de confirmar: si este enfoque te sirve tal cual, o si queres que compare alguna otra alternativa antes de pasar al Dia 3.

Dia 3 - Especificacion funcional

Actores: Conductor y Pasajero, ambos son el mismo tipo de entidad (PerfilRutix, ver Dia 1): un perfil actua como uno u otro segun el viaje.

Requisitos funcionales

ID	Requisito	Actor
RF1	Alta, baja, modificacion y consulta de perfil, con verificacion de identidad en dos niveles (DNI / DNI+licencia)	Conductor y Pasajero
RF2	Alta, baja, modificacion y listado de vehiculos propios	Conductor
RF3	Alta, baja y modificacion de medios de cobro: tarjetas y cuentas bancarias	Conductor y Pasajero
RF4	Publicar un viaje: fecha, horarios, origen, destino, vehiculo propio, asientos, valor por asiento, si pide sena	Conductor
RF5	Buscar y listar viajes por origen, destino, fecha y estado	Pasajero
RF6	Reservar uno o mas asientos en un viaje	Pasajero
RF7	Confirmar o rechazar una reserva pendiente	Conductor
RF8	Cancelar una reserva propia mientras este pendiente	Pasajero
RF9	Pagar una reserva (efectivo, transferencia o Mercado Pago)	Pasajero
RF10	Calificar el viaje al finalizar (puntaje de la reserva, valoracion del perfil)	Conductor y Pasajero
RF11	Iniciar el viaje (pasa de CREADO a EN_CURSO) y notificar a todos los pasajeros con reserva. Relevado como caso de uso, no se implementa en este TP.	Conductor

Reglas de negocio (confirmadas en el codigo existente)

ID	Regla
RN1	La fecha de salida de un viaje no puede ser anterior a la fecha actual
RN2	El vehiculo usado en un viaje debe pertenecer al perfil del conductor que lo publica
RN3	La cantidad de asientos reservados no puede superar los asientos disponibles del viaje
RN4	El conductor de un viaje no puede reservar asientos en su propio viaje
RN5	Un pasajero tiene como maximo una reserva por viaje: si vuelve a reservar, se suman los asientos a su reserva existente en vez de crear una nueva
RN6	Una reserva solo puede confirmarse, rechazarse o cancelarse si esta en estado PENDIENTE
RN7	Al rechazar o cancelar una reserva, sus asientos vuelven a estar disponibles en el viaje
RN8	No se puede eliminar una reserva que ya esta CONFIRMADA
RN9	Solo se puede pagar una reserva en estado CONFIRMADA (RegistrarPagoDeReservaComando, actualizado fuera del TP, ver Expediente Dia 3-4); el importe se calcula automaticamente (valor del viaje x asientos reservados), no lo define quien paga
RN10	Solo el pasajero dueno de la reserva puede registrar su pago
RN11	El CBU de una cuenta bancaria debe ser unico en el sistema; el token de una tarjeta tambien debe ser unico

Casos de uso principales

1. Publicar viaje (conductor): completa origen, destino, fecha/horarios, vehiculo y valor; el sistema valida RN1 y RN2. El viaje nace en estado CREADO.

2. Buscar y reservar (pasajero): filtra por origen/destino/fecha, elige un viaje y reserva asientos; el sistema valida RN3 y RN4.
3. Confirmar/rechazar reserva (conductor): revisa las reservas pendientes de su viaje y decide; el sistema valida RN6 y aplica RN7 si rechaza.
4. Pagar reserva (pasajero): paga la reserva confirmada, con el medio elegido; el sistema valida RN9 y RN10.
5. Cancelar reserva (pasajero): cancela mientras siga pendiente; el sistema valida RN6 y aplica RN7.
6. Iniciar viaje (conductor) - ya implementado (`IniciarViajeComando`): el conductor marca manualmente el inicio del recorrido (el viaje pasa de `CREADO` o `COMPLETO` a `EN_CURSO`) en el momento en que comienza a pasar a buscar a los pasajeros. Pendiente: la notificación a todos los pasajeros con reserva en ese viaje.
7. Completar viaje - implementado en dos vías: automática, dentro de `confirmar reserva`, cuando la suma de asientos de las reservas `CONFIRMADA` (incluyendo la que se acaba de confirmar) llega al total de asientos del viaje y este sigue en `CREADO`; y manual, mediante un botón del conductor en el frontend (`CompletarViajeComando`) para cerrar el viaje a nuevas reservas e iniciarlo más adelante aunque todavía queden asientos disponibles.

Criterios de aceptacion (ejemplos, verificables con la coleccion de Postman)

- Publicar un viaje con un vehiculo que no es del perfil logueado devuelve error (RN2).
- Reservar mas asientos que los disponibles devuelve error (RN3).
- El conductor de un viaje no puede registrar una reserva en ese mismo viaje (RN4).
- Confirmar dos veces la misma reserva devuelve error la segunda vez (RN6).
- Cancelar una reserva PENDIENTE libera los asientos; el viaje vuelve a mostrarlos disponibles (RN7).
- Pagar una reserva que no sea del propio pasajero devuelve error (RN10).

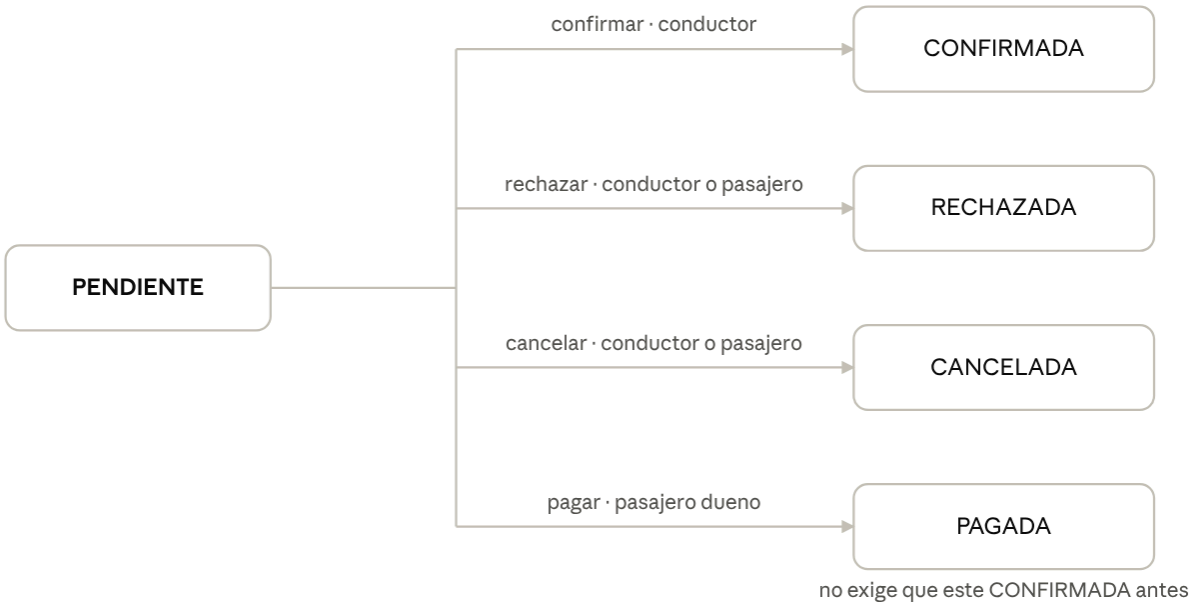
> Hallazgos a validar (comportamiento actual del codigo, no decisiones de diseno): pagar una reserva no exige que el conductor la haya confirmado antes (solo que este PENDIENTE); y las reglas PerfilConVehiculoHabilitadoReglaNegocio y PerfilConCuentaHabilitadaReglaNegocio existen pero su validacion real esta comentada (no hacen nada todavia). Quedan anotados para resolver en el Dia 9 (red-team). El hallazgo sobre el estado inicial del viaje ya se resolvio: ahora nace en CREADO (ver caso de uso 6, Iniciar viaje) y la transicion a EN_CURSO queda relevada como caso de uso futuro, fuera del alcance de implementacion de este TP.

Día 4 — Modelado de lógica: estados y flujo de Viaje, Reserva y Pago

Esta sección traza, a partir del código real (los `Comando` de `Viajes/comandos` y `Viajes/comandos/reservas`), qué transiciones de estado existen hoy, quién puede dispararlas y qué caminos alternativos (o ausencias) hay.

Reserva: el estado con más ramas

Una reserva se puede pagar sin haber sido confirmada antes

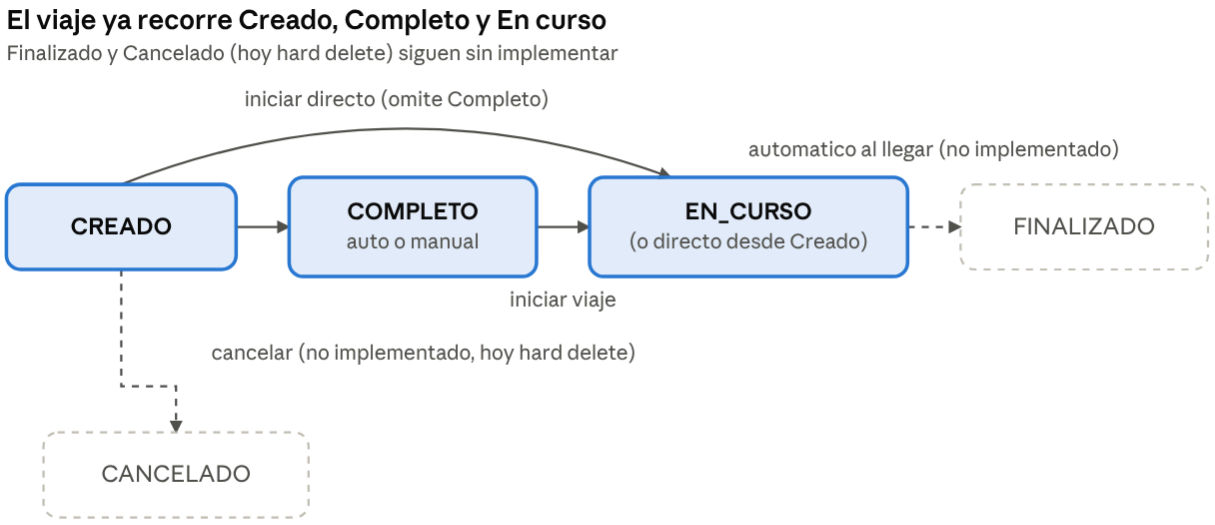


estados de la Reserva · 5 estados, 1 brecha

La `Reserva` es la entidad con más lógica de transición implementada. `ConfirmarReservaDeViajeComando` exige que quien confirma sea el conductor del viaje. `RechazarReservaDeViajeComando` y `CancelarReservaDeViajeComando` aceptan tanto al conductor como al pasajero dueño de la reserva, y en ambos casos liberan los asientos ocupados. Pero `RegistrarPagoDeReservaComando` no valida que la reserva esté `CONFIRMADA`: una reserva en `PENDIENTE` puede pagarse directamente. Es una decisión de diseño a confirmar, no necesariamente un error, pero hoy no está documentada ni validada explícitamente en el código.

Además, `ActualizarReservaDeViajeComando` expone un `PUT` genérico que setea `estado` tal cual venga en el DTO, sin pasar por ninguna de las reglas anteriores: en la práctica, bypassea por completo la máquina de estados de la Reserva (RN6).

Viaje: una sola transición real hoy



estados del Viaje · Creado, Completo y En curso ya implementados

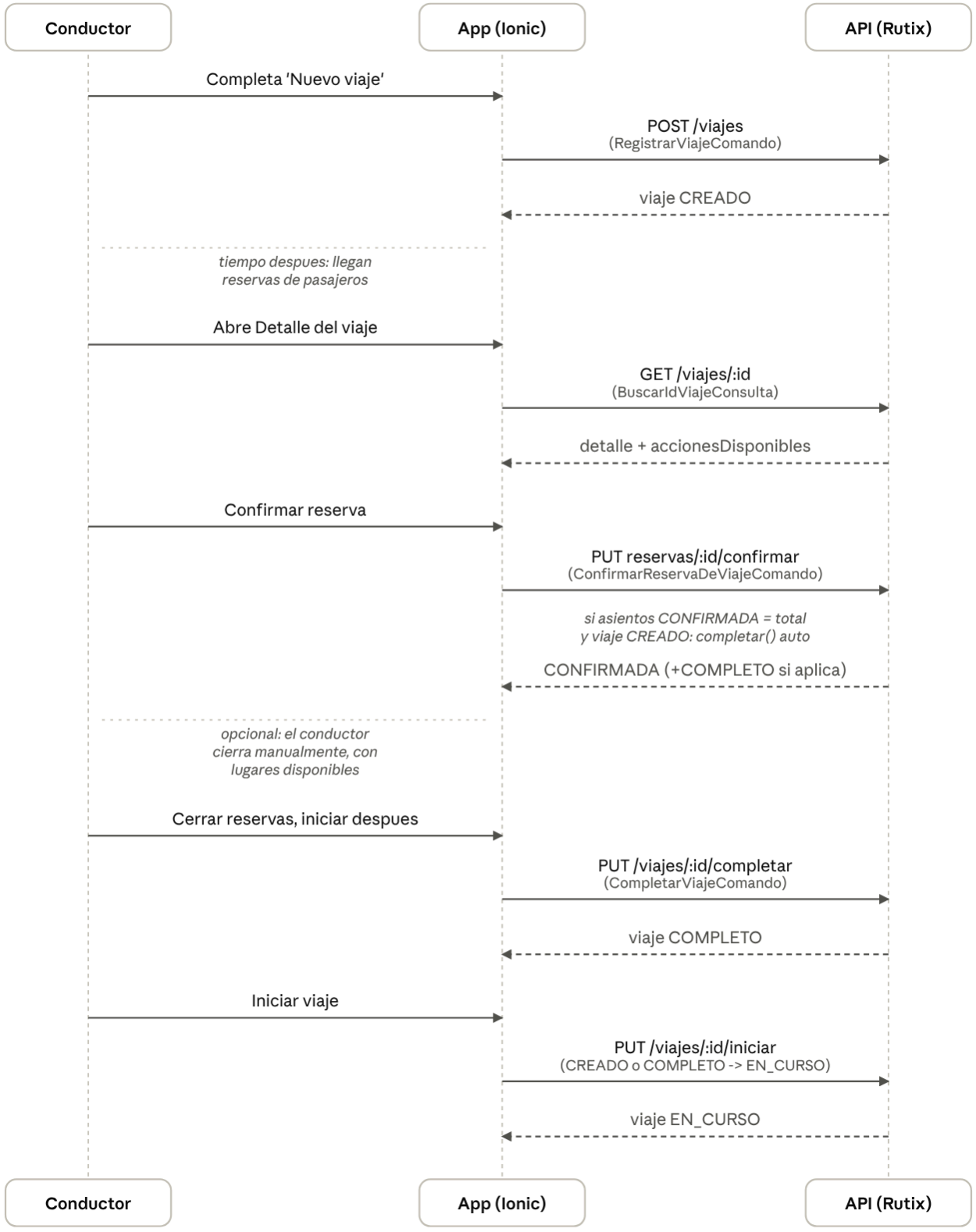
Hoy sí hay comandos dedicados para avanzar el estado del `Viaje`. `RegistrarViajeComando` lo crea en `CREADO` (decisión del Día 3). `Viaje.confirmarReserva(id)` confirma la reserva y, si la suma de asientos de las reservas `CONFIRMADA` llega al total de asientos del viaje y el viaje sigue en `CREADO`, lo completa automáticamente (pasa a `COMPLETO`). El conductor también puede completar el viaje a mano, con asientos aún disponibles, invocando `CompletarViajeComando` desde un botón del frontend ("Cerrar reservas e iniciar más adelante"). `IniciarViajeComando` ya es real: acepta el viaje en `CREADO` o en `COMPLETO` y lo pasa a `EN_CURSO`. `ActualizarViajeComando` sigue siendo, igual que en Reserva, un `PUT` genérico sin validación de transición. `EliminarViajeComando` sigue borrando el registro directamente (hard delete), sin pasar por un estado `CANCELADO`.

Esto resuelve el caso de uso relevado como pendiente en el Día 3: "iniciar viaje" (marcarlo `EN_CURSO` cuando el conductor empieza a buscar pasajeros) es RF11 y ya tiene implementación real en `IniciarViajeComando`. Lo que sigue sin implementar es la notificación a los pasajeros con reserva en ese viaje, y las transiciones a `FINALIZADO` y `CANCELADO` (ver Día 3, caso de uso 7).

Secuencia del viaje: conductor y pasajero

Flujo del viaje: conductor

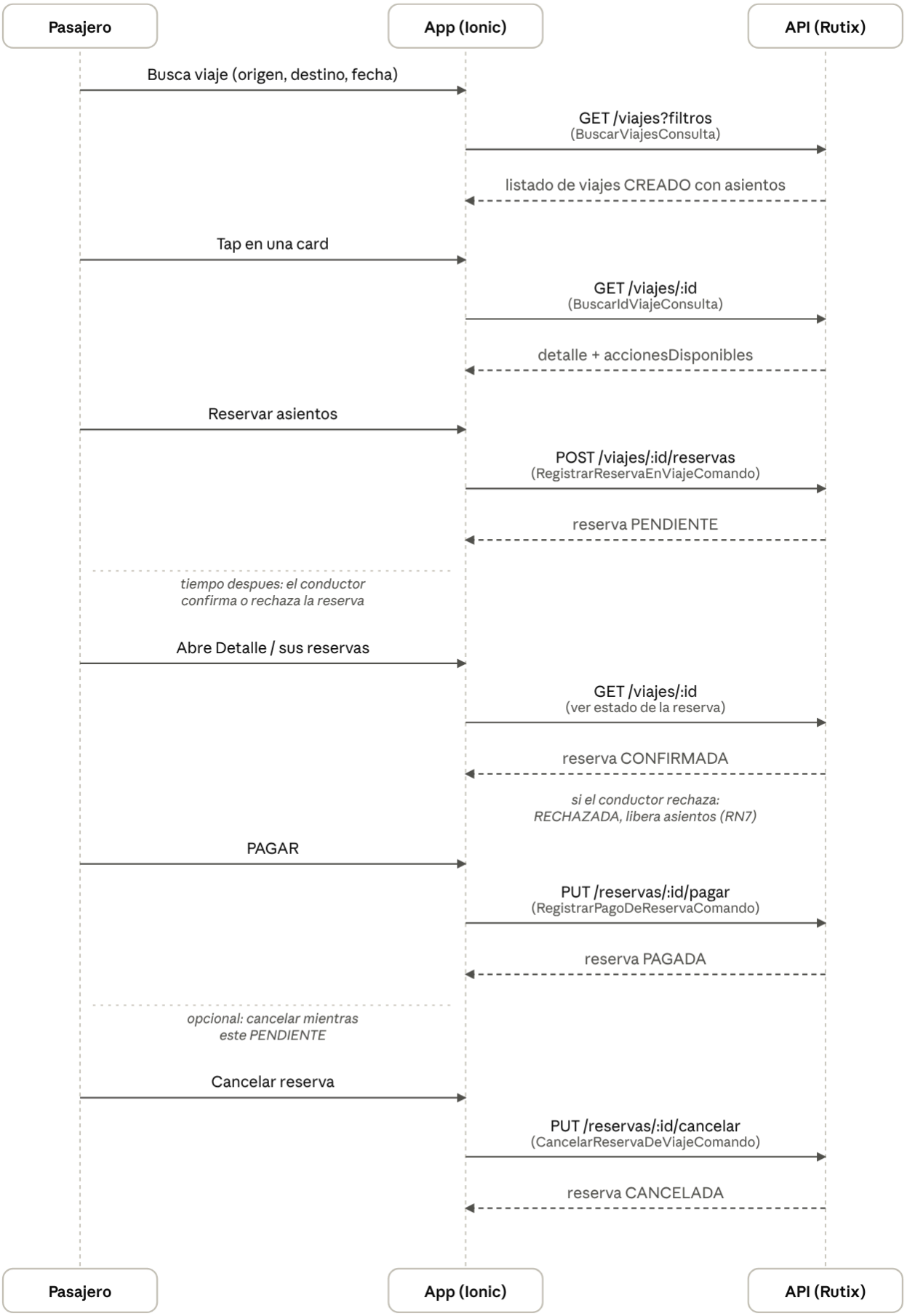
Rutix · publicar, confirmar reservas, completar (auto o manual) e iniciar



secuencia del viaje · conductor: publicar, confirmar, completar e iniciar

Flujo del viaje: pasajero

Rutix · buscar, reservar, esperar confirmacion, pagar o cancelar



secuencia del viaje · pasajero: buscar, reservar, pagar o cancelar

Ambos diagramas usan los comandos reales de Viaje y Reserva (RegistrarViajeComando, ConfirmarReservaDeViajeComando, CompletarViajeComando, IniciarViajeComando, RegistrarReservaEnViajeComando, RegistrarPagoDeReservaComando, CancelarReservaDeViajeComando); conductor y pasajero convergen en el mismo Detalle del viaje (Día 7).

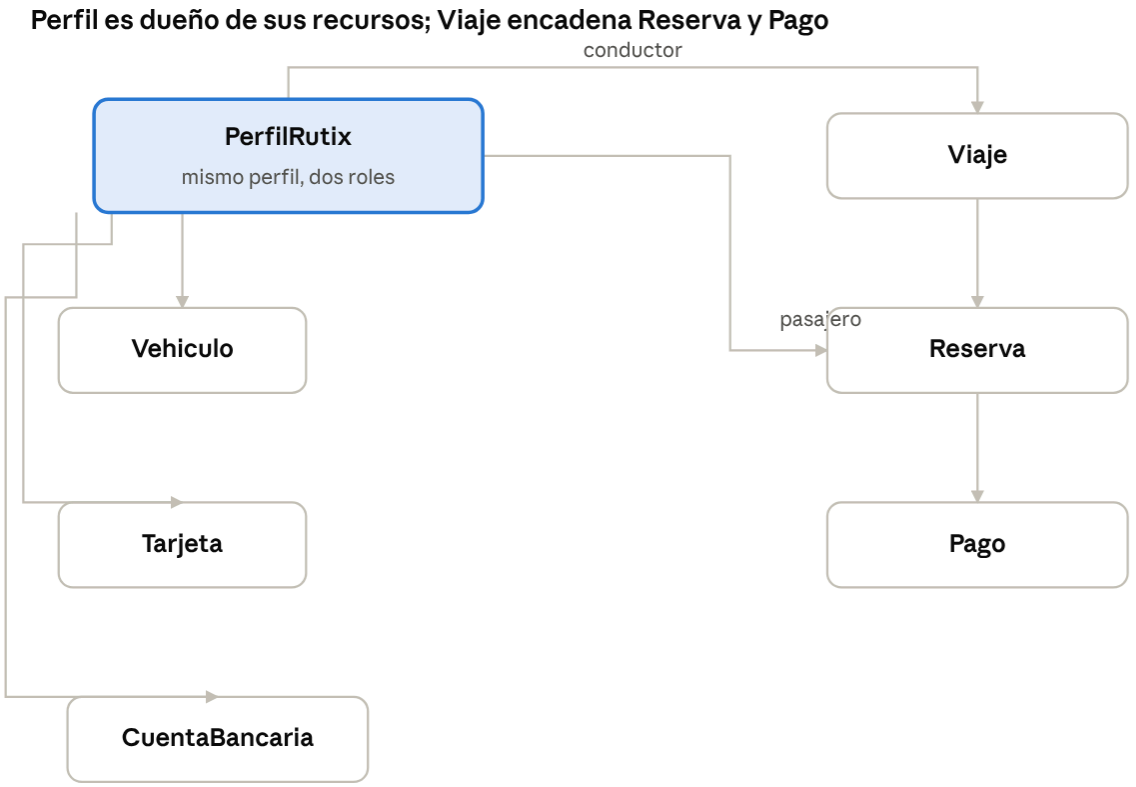
Pago: sin máquina de estados en uso

`Pago` se crea siempre en `PENDIENTE`. Existen los valores `CONFIRMADO` y `ERROR` en `EstadoPagoEnum`, pero ninguna estrategia de cobro (`EfectivoCobroStrategy`, `MercadoLibreCobroStrategy`) los asigna todavía: son stubs. No se armó un diagrama de estados para Pago porque en el código actual no hay transición real que mostrar, es un único estado fijo.

Hallazgo para el expediente

`ActualizarViajeComando` y `ActualizarReservaDeViajeComando` son endpoints `PUT` genéricos que permiten cambiar el `estado` sin pasar por las reglas de negocio específicas (RN6 y análogas). Es una brecha real entre el modelo de estados documentado y lo que el código permite hacer vía API. Queda anotado como hallazgo para la revisión de seguridad / red-team (Día 9)`. Esa decisión ya se tomó y se aplicó fuera del TP: pagar exige CONFIRMADA (ver RN9 del Día 3 y Expediente, fila Dia 3-4)`.

Día 5 — Datos y persistencia: modelo de datos y justificación de la base elegida



modelo de datos de Rutix · 7 entidades, Perfil como eje

El diagrama muestra la forma real del esquema: `PerfilRutix` es la entidad central — el mismo perfil actúa como conductor de un `Viaje` o como pasajero de una `Reserva`, sin tablas de rol separadas. A su vez es dueño de sus propios `Vehiculo`, `Tarjeta` y `CuentaBancaria` (relación 1-N con `cascade = ALL` y `orphanRemoval = true`: si se borra el perfil, se borran sus vehículos, tarjetas y cuentas). El dominio de viajes es una cadena lineal: `Viaje` 1-N `Reserva` 1-N `Pago`.

Entidad	Relación clave	Notas de persistencia
PerfilRutix	1-N Vehiculo, Tarjeta, CuentaBancaria	`usuarioId` es un `Long` suelto, sin relación JPA: la cuenta de login vive en otro microservicio del ecosistema, no en Rutix
Viaje	N-1 PerfilRutix (conductor), N-1 Vehiculo, 1-N Reserva	`listaReservas` con `fetch = EAGER`
Reserva	N-1 PerfilRutix (pasajero), N-1 Viaje, 1-N Pago	`pasajero` con `fetch = LAZY`; `listaPagos` con `fetch = LAZY`

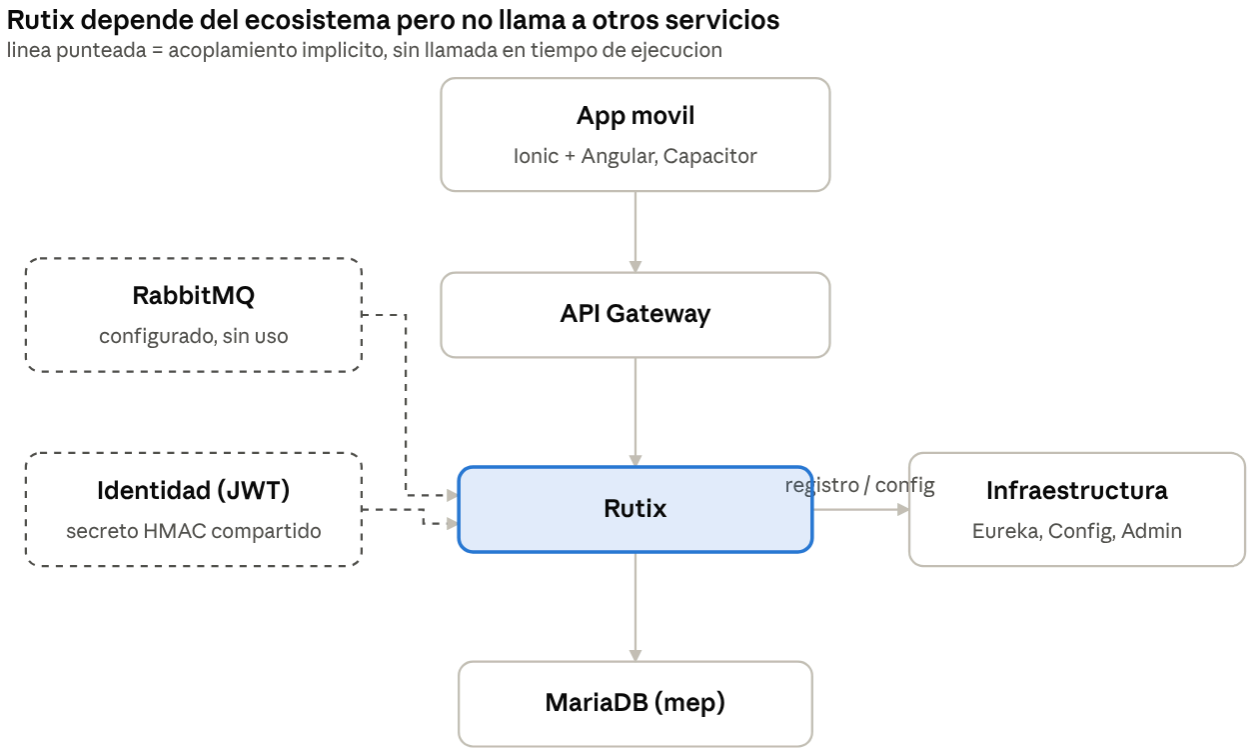
Entidad	Relación clave	Notas de persistencia
Pago	N-1 Reserva	se crea siempre en `PENDIENTE` (ver Día 4)

Por qué una base relacional. El proyecto usa MariaDB (`jdbc:mariadb://.../mep` , dialecto `MariaDBDialect`). Encaja con el dominio por tres razones concretas, visibles en el código: hay integridad referencial real que importa (una `Reserva` no existe sin `Viaje` , un `Pago` no existe sin `Reserva` , reforzado con `cascade = ALL` y `orphanRemoval = true` , no solo convención de nombres); las consultas centrales son relacionales (buscar viajes por ciudad y fecha, listar reservas de un viaje, calcular el total pagado de una reserva vía `getValorTotalReserva()` , que multiplica `viaje.valor x asientos`); y el volumen y la concurrencia de escritura no piden un modelo distribuido o eventualmente consistente — los asientos y los pagos necesitan consistencia inmediata (ver RN3, RN9 del Día 3).

El esquema `mep` es compartido por todos los microservicios del ecosistema en la misma instancia MariaDB (ver `application-local.yml`), pero cada uno gestiona sus propias tablas sin relaciones JPA cruzadas entre servicios: Rutix no tiene una tabla ni una entidad `Usuario` , solo guarda su id. Es el mismo principio de *database per service* que ya eligió la arquitectura de microservicios del Día 1, aplicado a nivel de esquema aunque no de instancia física de base de datos.

> Hallazgos para el expediente (ambos resueltos fuera del TP, ver Expediente Día 5): (1) `ddl-auto: update` dejaba que Hibernate generara y actualizara el esquema automaticamente a partir de las entidades, sin migraciones versionadas — se agrego Flyway (baseline V1) y `ddl-auto` paso a `validate` . (2) Las colecciones de `PerfilRutix` (`listaVehiculos` , `listaTarjetas` , `listaCuentas`) y de `Viaje` (`listaReservas`) usaban `fetch = EAGER` — se confirmo por grep que ningun listado paginado las necesitaba cargadas de entrada y se cambiaron a `LAZY` .

Día 6 — Arquitectura e integraciones: diagrama de componentes y contratos



arquitectura de Rutix · 1 llamada entrante real, 2 acoplamientos implícitos

Rutix es un microservicio pasivo: no tiene ningún `@FeignClient` ni llama por HTTP a otro servicio del ecosistema (se buscó en todo el código, no hay ninguno). Todo lo que recibe entra por `ViajeController`, `ReservaController`, `PerfilRutixController`, `VehiculoController`, `TarjetaController` y `CuentaBancariaController`, detrás del API Gateway, que a su vez lo resuelve consultando a Eureka. El `GlobalExceptionHandler` centraliza el formato de error de los seis controladores.

Componente	Rol en Rutix	Estado real en el código
API Gateway + Eureka	enruta las requests del app móvil hacia esta instancia	Rutix se registra como cliente Eureka (`eureka.client.registerWithEureka: true`)
Config Server	le entrega su configuración al arrancar	`spring-cloud-starter-config` + `spring-cloud-starter-bootstrap` en el `pom.xml`
Spring Boot Admin	lo monitorea como cliente	`spring-boot-admin-starter-client` registrado contra `localhost:8086`
MariaDB (`mep`)	persistencia (Día 5)	única dependencia de datos real

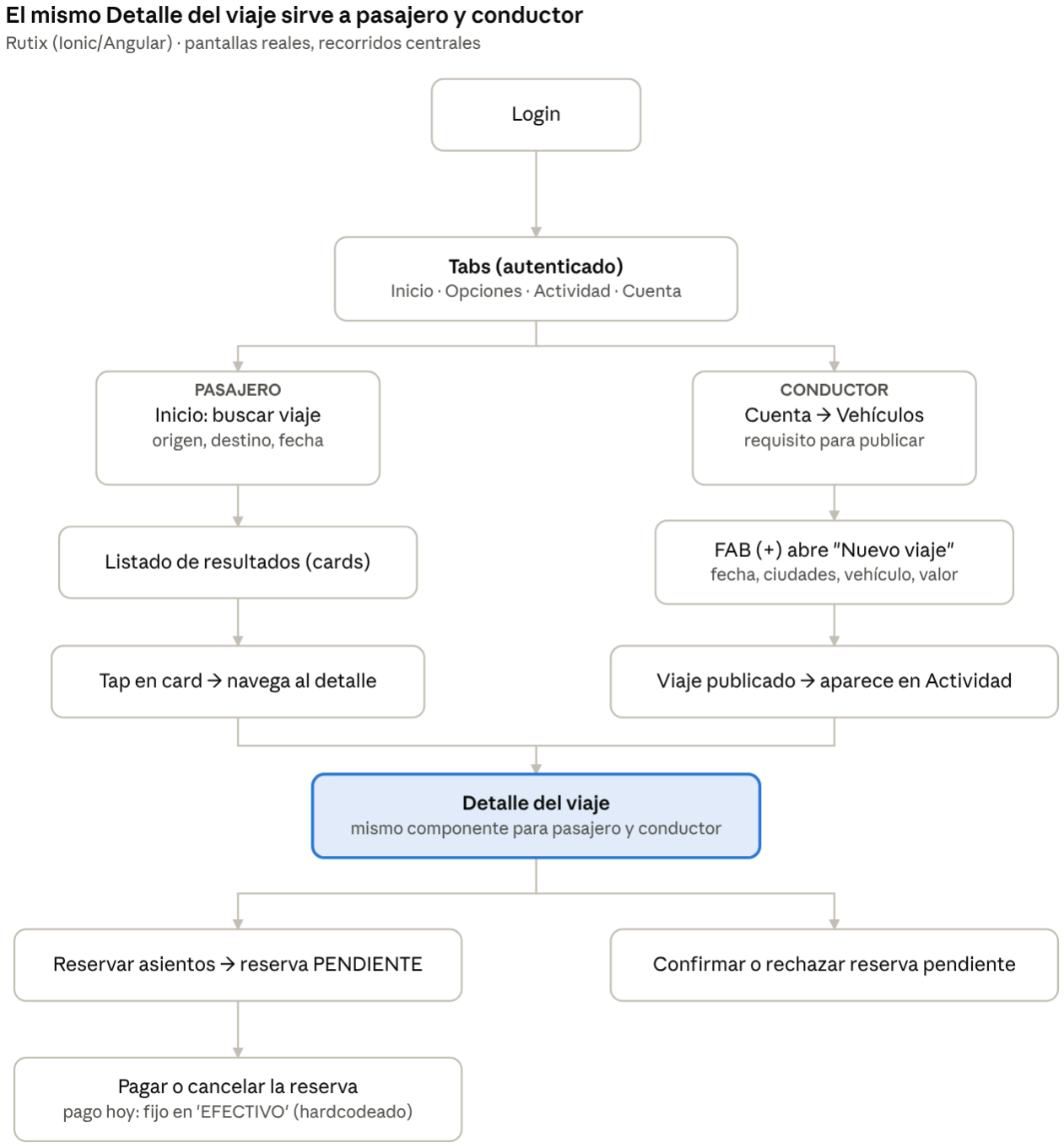
Componente	Rol en Rutix	Estado real en el código
RabbitMQ	mensajería asíncrona	**configurado pero sin usar**: tiene host/puerto en `application-local.yml` y la dependencia `spring-boot-starter-amqp`, pero no hay ningún `@RabbitListener` ni uso de `RabbitTemplate` en todo el proyecto
Identidad (JWT)	autenticación	Rutix no llama a un servicio de login: valida (y hasta puede firmar) tokens con un secreto HMAC compartido (`security.jwt.key.private`), confiando en que quien lo emitió use el mismo secreto

El contrato real con el resto del ecosistema es un secreto compartido, no una llamada. `JwtTokenFilter` lee el header `Authorization`, decodifica el JWT con `JwtUtils.validateToken()` y arma la `Authentication` de Spring Security a partir de sus claims (`username`, `authorities`) — sin ninguna consulta de red a un servicio de usuarios. El único acoplamiento con la identidad del ecosistema es que ese secreto HMAC tiene que ser idéntico en todos los servicios que emiten o validan tokens; no hay una relación cliente-servidor explícita, es confianza por configuración compartida.

Verificado contra el código real de `ms-security` y `config-service` (proyecto hermano `mep-services`, fuera de Rutix): la clave no es solo "idéntica en todos los servicios" en abstracto — está literalmente copiada carácter por carácter, en texto plano, en los `application-local.yml` de Rutix y de ms-security, ambos versionados en sus respectivos repositorios. `AuthenticationController` (`ms-security`) expone `POST /v1/auth/log-in` y `/v1/auth/sign-up`, y `JwtUtils.createToken()` firma con `Algorithm.HMAC256(privateKey)` usando esa misma clave. Rutix además declara en su `bootstrap.yml` un `spring.cloud.config.uri` apuntando a `ms-config:8081` (con usuario/clave también en texto plano), pero `config-data/` — el backend *native* del config-service — no tiene ningún `rutix.yml`: la integración con el config-service está declarada pero no tiene nada que entregarle a Rutix, que en la práctica sigue funcionando enteramente con sus valores locales hardcodeados.

> Hallazgo para el expediente (resuelto parcialmente fuera del TP, ver Expediente Día 6): `SecurityConfig` tenia `anyRequest().permitAll()` — se cambio a `authenticated()` (con excepcion de `/actuator`, `/swagger-ui` y `/v3/api-docs`), y `CustomAuthenticationEntryPoint` ahora devuelve 401 explicito en vez del 403 vacio por defecto. Sigue pendiente: la clave en `security.jwt.key.private` es una clave RSA pegada como string pero usada con `Algorithm.HMAC256` (simetrico) — un secreto compartido de texto plano, ademas duplicado caracter por caracter entre Rutix y ms-security (ver H10 y H21 en el backlog).

Día 7 — Interfaz y experiencia: recorridos centrales



recorridos de Rutix (Ionic) · pasajero y conductor convergen en Detalle del viaje

Rutix (Ionic standalone, 4 pestañas detrás de `authGuard`: Inicio, Opciones, Actividad, Cuenta) ya tiene las pantallas reales de los dos recorridos centrales. Como pasajero: `Inicio` busca por ciudad de partida, destino y fecha (`BusquedaViajesFormComponent`) y lista resultados en `ListadoViajesGrillaComponent`; tocar una `ViajeCard` navega a `/tabs/actividad/viajes/detalle/:id`, donde se reserva una cantidad de asientos (`registrarReserva()`) y aparece una tarjeta de la propia reserva con **Pagar** o **Cancelar**.

Como conductor: publicar un viaje requiere tener al menos un vehículo cargado en `Cuenta → Vehículos`; desde `Inicio`, el FAB (+) abre el modal "Nuevo viaje" (fecha, horarios, ciudades, vehículo, asientos, valor y seña opcional). El viaje publicado aparece en `Actividad` bajo el segmento "Conduciendo", y el mismo `ViajeDetallePage` le muestra el listado de reservas con **Confirmar**/**Rechazar** cuando hay pendientes.

`ViajeDetallePage` es un único componente para los dos roles: qué botones se muestran depende de `viaje.accionesDisponibles` (un campo que devuelve el backend), leído por `puedeReservar()` y `puedeGestionarReserva()` — no hay un chequeo de rol hardcodeado en el front.

Pantalla	Ruta	Acciones reales
Inicio	`/tabs/inicio`	Buscar viajes, ver listado, publicar viaje (FAB)
Actividad	`/tabs/actividad`	Listar "mis viajes", segmento Conduciendo/Pasajero
Detalle del viaje	`/tabs/actividad/viajes/detalle/:id`	Reservar, pagar, cancelar (pasajero) · confirmar, rechazar (conductor)
Cuenta	`/tabs/cuenta`	Perfil, billetera, vehículos

> **Hallazgo (nuevo, Alta):** `reserva-card.component.html` muestra el botón **PAGAR** solo cuando la reserva está en estado `PENDIENTE` — pero `RegistrarPagoDeReservaComando` (actualización de Día 3-4) ahora exige `CONFIRMADA`. Tal como está hoy el front, un pasajero vería el botón de pago antes de que el conductor confirme, y el backend lo rechazaría: front y back quedaron desincronizados por ese cambio. Lo agrego al backlog como **H14** (Alta) — resuelto fuera del TP: se corrigio el *ngIf de reserva-card.component.html para depender de CONFIRMADA en vez de PENDIENTE. Nota menor: `ViajeFormDetalleReservarComponent` y `ViajeFormDetalleConsultarComponent` están construidos pero no conectados — la reserva real se hace inline en `ViajeDetallePage` — lo agrego como **H13** (Baja, código muerto).

Día 8 — Seguridad y validación: riesgos, permisos y criterios de aceptación

Rutix no define roles de Spring Security (no hay ningún `@PreAuthorize`, `hasRole` ni `hasAuthority` en todo el backend): el filtro JWT solo establece *quién* está autenticado, y es cada `Comando` quien decide a mano, línea por línea, si ese usuario puede tocar el recurso. Esta sección releva ese modelo contra el código real y documenta dónde el chequeo falta.

Autenticación: JWT stateless

`SecurityConfig` exige `authenticated()` en toda ruta salvo `/actuator/**`, `/v3/api-docs/**` y `/swagger-ui/**`; sin sesión (`STATELESS`), sin `formLogin` ni `httpBasic`. `JwtTokenFilter` lee el header `Authorization`, valida el JWT con `JwtUtils` (HMAC256 sobre `security.jwt.key.private`) y reconstruye el `Authentication` con el `userId` como subject y las `authorities` del claim; si el token viene pero es inválido, corta ahí mismo con 401 explícito. Si no viene token, deja pasar la request sin `Authentication`, y es `CustomAuthenticationEntryPoint` quien la rechaza más adelante con 401 (reemplaza el 403 vacío por defecto de Spring Security).

Autorización: patrón declarativo y manual, con bypass de admin

Cada `Comando` que escribe un recurso debería comparar el `PerfilRutix` autenticado (`SecurityContextHolder`) contra el dueño real del recurso, salvo que el usuario autenticado tenga el rol `ROLE_ADMIN` (emitido por `ms-security`), en cuyo caso el chequeo de dueño se bypasea. Hoy el patrón se aplica, con ese bypass, en los nueve `Comando` que escriben sobre Viaje, Reserva o datos financieros: cinco vía `@PreAuthorize` declarativo en el controller (beans `ViajeSecurity`, `ReservaSecurity`, `PerfilRecursoSecurity`) y cuatro vía chequeo manual dentro del propio `Comando` (`RolUtils.esAdmin`):

Comando	Chequea dueño	Patrón
ConfirmarReservaDeViajeComando / RechazarReservaDeViajeComando / CompletarViajeComando / IniciarViajeComando	Sí	conductor del viaje, o admin (`RolUtils.esAdmin`). `RechazarReservaDeViajeComando` comparaba `Long` con `!=` en el chequeo de dueño (mismo bug que H20): corregido a `.equals()` (H22)
ActualizarReservaDeViajeComando	Sí	pasajero dueño de la reserva, solo en PENDIENTE, o admin (`RolUtils.esAdmin`)

Comando	Chequea dueño	Patrón
RegistrarPagoDeReservaComando	Sí, bug corregido	pasajero dueño, comparaba `Long` con `!=` en vez de `.equals()` (corregido, H1/H20)
ActualizarVehiculoComando / EliminarVehiculoComando	Sí, estructural	opera sobre el `PerfilRutix` autenticado, nunca recibe un id ajeno; no tiene hoy una vía de bypass para admin (limitación de H23)
ActualizarViajeComando	Sí	`@PreAuthorize` con `@viajeSecurity.esConductor`, o admin (H15, resuelto)
EliminarViajeComando	Sí	`@PreAuthorize` con `@viajeSecurity.esConductor`, o admin (H16, resuelto)
EliminarReservaDeViajeComando	Sí	`@PreAuthorize` con `@reservaSecurity.esPasajeroDueno`, o admin (H17, resuelto)
ActualizarCuentaBancariaComando	Sí	`@PreAuthorize` con `@perfilRecursoSecurity.esDuenoDeCuenta`, o admin (H18, resuelto)
ActualizarTarjetaComando	Sí	`@PreAuthorize` con `@perfilRecursoSecurity.esDuenoDeTarjeta`, o admin (H19, resuelto)

Validación de datos

Los DTO de alta (`RegistrarViajeRequestComandoDto`, `RegistrarReservaDto`, etc.) usan Bean Validation (`@Valid` en el Controller); `MethodArgumentNotValidException` se mapea en `GlobalExceptionHandler` a 400 con la lista de errores por campo. Validadores propios: `ValidFechaSalidaViaje`, `ValidPerfilRutixId`, `ValidViajeId`. Reglas de negocio con nombre propio (`TokenUnicoReglaNegocio`, `CbuUnicoReglaNegocio`, `PerfilConVehiculoHabilitadoReglaNegocio`, etc.) se aplican a mano dentro de cada `Comando`. Los `PUT` / `DELETE` genéricos de la tabla anterior no llevan ninguna de estas reglas: ni de propiedad ni de negocio.

Hallazgos de seguridad (nuevos)

> **IDOR en Viaje, Reserva y datos financieros (Alta).** `ActualizarViajeComando`, `EliminarViajeComando`, `EliminarReservaDeViajeComando`, `ActualizarCuentaBancariaComando` y `ActualizarTarjetaComando` operan sobre un id recibido del cliente sin comparar contra el usuario autenticado: cualquier usuario logueado puede editar o borrar un viaje ajeno (incluso reasignar su conductor), borrar una reserva ajena, o reescribir el CBU o el token de una tarjeta de otro perfil. Contrasta con el patrón correcto ya usado en `ActualizarVehiculoComando` / `EliminarVehiculoComando` (operan siempre sobre el `PerfilRutix` autenticado) y en `ActualizarReservaDeViajeComando` (compara el pasajero dueño a mano). Agregados al backlog como H15–H19. Resuelto: se agregó @PreAuthorize con beans de autorización por recurso (ViajeSecurity, ReservaSecurity, PerfilRecursoSecurity) sobre estos cinco Comando; backlog marcado Resuelto (ver Expediente, fila "Día 8 (actualización)").
>
> **Comparación de `Long` con `!=` en `RegistrarPagoDeReservaComando` (Media).** `usuarioLogueado.getId() != usuarioReserva.getId()` compara referencias, no valores: para la mayoría de los ids reales (fuera del cache de `Long`, -128 a 127) la comparación da `true` aunque el id sea el mismo, por lo que el chequeo puede bloquear al dueño legítimo del pago en vez de solo bloquear a un tercero. Falla cerrado (deniega de más), no es una brecha de seguridad, pero rompe el flujo de pago real. Agregado al backlog como H20. Resuelto: se corrigió a .equals() en el mismo cambio; backlog marcado Resuelto.

Criterios de aceptación

Criterio	Estado
Toda request sin JWT válido recibe 401 explícito, nunca 403 vacío ni 500	Cumplido (`CustomAuthenticationEntryPoint`, `JwtTokenFilter`)
Toda escritura sobre un Viaje valida que el usuario autenticado es su conductor	Cumplido (`@PreAuthorize` con `@viajeSecurity.esConductor`, o admin; H15/H16 resueltos)
Toda escritura sobre una Reserva valida que el usuario autenticado es su pasajero (o el conductor del viaje, segun la accion)	Cumplido (`@PreAuthorize` con `@reservaSecurity.esPasajeroDueno`, o admin; H17 resuelto)
Toda escritura sobre Vehiculo, Tarjeta o CuentaBancaria valida que pertenecen al perfil autenticado	Cumplido en los tres (H18/H19 resueltos con @PreAuthorize, o admin; Vehiculo ya operaba estructuralmente sobre el perfil autenticado)

Criterio	Estado
Los errores de validacion devuelven 400 con el detalle por campo	Cumplido (`MethodArgumentNotValidException` -> `GlobalExceptionHandler`)
El secreto usado para firmar el JWT no se expone en texto plano en el repositorio	Incumplido (hallazgo del Dia 6, aun pendiente)

La brecha mas seria — los `PUT` / `DELETE` genericos sin chequeo de dueño — se agrega al backlog para resolverse fuera de este TP; dado el alcance academico, este Dia 8 se centra en relevarla y documentarla, no en corregir el codigo real (a diferencia de otros dias, donde si se aplico el fix de inmediato). Actualizacion: a pedido del usuario, el fix (@EnableMethodSecurity + @PreAuthorize con beans de autorizacion por recurso, y la correccion del bug de Long) se implemento despues en el backend real; ver Expediente, fila "Dia 8 (actualizacion)", y H15-H20 en el backlog, ya marcados Resuelto.

Día 9 — Red-team y reconciliación final

Los Días 3 a 7 fueron dejando hallazgos y decisiones marcadas explícitamente como "a revisar en el Día 9", a medida que el código real avanzaba en paralelo al TP. Este día cierra esa deuda: reconcilia el texto de esos días con el estado actual del código, y agrega una revisión adversarial (red-team) sobre lo que todavía queda abierto.

Reconciliación de lo diferido

- **RN9 (Día 3)** y el caso de uso "Pagar reserva": decían que se podía pagar en PENDIENTE o CONFIRMADA. Ya está corregido en el documento — hoy `RegistrarPagoDeReservaComando` exige CONFIRMADA (H1, resuelto fuera del TP).
- **Migraciones y fetch EAGER (Día 5)**: los dos hallazgos del expediente del Día 5 quedaron marcados resueltos en el texto — Flyway con baseline versionado, y las cuatro colecciones pasadas a LAZY.
- **`permitAll()` (Día 6)**: el hallazgo quedó marcado resuelto — `SecurityConfig` exige `authenticated()`. El secreto HMAC en texto plano sigue abierto (H10), y ahora con más detalle: está duplicado carácter por carácter entre Rutix y `ms-security` (H21).
- **Hallazgo nuevo del Día 7** (botón PAGAR mostrado en PENDIENTE): quedó marcado resuelto — el frontend ya depende de CONFIRMADA (H14).

Lo que el red-team encuentra todavía abierto

PUT genéricos sin máquina de estados (H3, H4 — siguen Pendiente). `ActualizarReservaDeViajeComando` ya exige ser el pasajero dueño y que la reserva esté PENDIENTE (Día 3-4, actualización), pero adentro sigue haciendo `r.setEstado(value.getEstado())` directo desde el DTO, sin pasar por `Viaje.confirmarReserva()` / `rechazarReserva()` / `cancelarReserva()`: un pasajero dueño de su propia reserva PENDIENTE podría mandar `estado: CONFIRMADA` él mismo. `ActualizarViajeComando` tiene el mismo problema del lado del Viaje (ya tiene `@PreAuthorize` de dueño/admin desde el Día 8, pero ninguna validación de transición de estado). Ninguno de los dos entró en el alcance del fix de seguridad del Día 8 porque ese día se enfocó en *quién* puede escribir, no en *qué* estados son válidos — son problemas distintos y H3/H4 quedan correctamente abiertos para una iteración futura sobre la máquina de estados.

Hallazgo nuevo — contador de asientos sin control de concurrencia (H24, Alta). `Viaje.asientosReservados` / `asientosDisponibles` son enteros simples (confirmado por grep en `Viaje.java`): no hay un campo `@Version` ni ningún otro mecanismo de bloqueo. `ActualizarReservaDeViajeComando`, `ConfirmarReservaDeViajeComando` y `RechazarReservaDeViajeComando` todos leen el `Viaje`, modifican esos contadores en memoria y guardan — un patrón clásico de *lost update*: dos pasajeros reservando el último asiento al mismo tiempo pueden pasar ambos la validación de "asientos disponibles" antes de que cualquiera de los dos guarde, y el viaje termina sobrevendido. No se corrige en este TP (es un cambio de modelo de datos, no solo de un `Comando`); se deja documentado para decidir entre `@Version` (optimistic locking) o una consulta atómica de decremento.

Día 10 — Consolidación y auditoría final

Resumen ejecutivo

Este TP usó a Rutix, un proyecto real en desarrollo, como caso de estudio: cada día partió del código tal como estaba (no de una maqueta) — requisitos y reglas de negocio extraídos de los `Comando` existentes, diagramas de estados y de secuencia trazados contra el comportamiento real, modelo de datos justificado contra las entidades JPA, arquitectura mapeada contra los controllers y `SecurityConfig`, y una auditoría de seguridad que encontró y, en varios casos, corrigió brechas reales. El proyecto también evolucionó en paralelo al TP (nuevos estados, nuevos comandos, fixes de seguridad), lo que obligó a mantener una práctica de reconciliación explícita en vez de dejar que la documentación se desalinee del código.

Estado final del backlog de hallazgos técnicos

De los 24 hallazgos registrados (H1–H24), **14 están Resueltos** y **10 siguen Pendientes**. Los pendientes de prioridad Alta, en orden de atención recomendado:

1. **H10 / H21 — secreto HMAC del JWT en texto plano, duplicado entre Rutix y `ms-security`.** Recomendación: mover a una variable de entorno o a un secret manager, no commitear la clave en ningún yml.
2. **H6 — reglas de habilitación por vehículo/cuenta comentadas, no hacen nada hoy.** Recomendación: decidir si se reactivan o se eliminan del código (código muerto que aparenta ser una validación real es peor que no tenerla).
3. **H24 — contadores de asientos sin `@Version`, riesgo de sobreventa por concurrencia.** Recomendación: agregar optimistic locking antes de exponer el viaje a más de un pasajero reservando en simultáneo.
4. **H3 — `ActualizarReservaDeViajeComando` sigue seteando `estado` directo del DTO.** Recomendación: eliminar el campo `estado` del DTO de actualización y forzar que todo cambio de estado pase por `confirmar()` / `rechazar()` / `cancelar()`.
5. **H3 (viaje) / nota de H4 — `ActualizarViajeComando` sin validación de transición de estado.** Misma recomendación que el punto anterior, aplicada a Viaje.

Los pendientes de Media (H4 duplicado con el punto 5, H5 hard delete de Viaje sin estado CANCELADO, H12 notificaciones no diseñadas) y de Baja (H11 RabbitMQ sin uso, H13 código muerto en el frontend) quedan para una iteración post-TP, sin riesgo de seguridad asociado.

Reflexión sobre el proceso de co-work

Lo que funcionó bien en este TP de "Co-Work 2027": cada afirmación de diseño se verificó contra el código real antes de escribirse (grep, lectura de archivos, captura de pantalla de los diagramas), y cada fix de seguridad se aplicó solo después de una decisión humana explícita ("implementalo ahora", "sí por favor"), nunca de forma unilateral. Cuando el código cambiaba en paralelo al día que se estaba escribiendo, se dejó constancia en el Expediente en vez de reescribir en silencio — eso permitió llegar al Día 9 con una lista concreta de qué reconciliar, en vez de descubrir inconsistencias de golpe.

La limitación más importante: este entorno no puede compilar ni ejecutar el backend de Rutix. Todos los fixes de seguridad (H15–H20, H22, H23) se aplicaron escribiendo los archivos directamente y verificándolos por `grep`, pero ninguno fue probado end-to-end por la IA. Eso quedó explícito cada vez, pero vale repetirlo acá: antes de llevar estos cambios a producción, hay que compilar Rutix y probar cada endpoint con un usuario dueño, uno ajeno y uno `ROLE_ADMIN`.

Próximos pasos

- Resolver los 5 hallazgos Alta pendientes, en el orden sugerido arriba.
- Compilar y correr el backend real para validar los fixes de seguridad del Día 8/9.
- Cuando el MVP de Rutix se considere cerrado, aplicar las recomendaciones de *UML y Patrones* (Larman) y *Código Limpio* pendientes de una revisión anterior del proyecto.

Expediente de co-work

Registro de los intercambios con IA mas relevantes del proyecto: objetivo, prompt, propuesta, decision humana y verificacion. Se completa dia a dia, junto con el diseno.

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 1	Delimitar el problema a partir del codigo real de Rutix	-	-	-	-	-	-
Dia 1	Confirmar alcance de la ficha del problema	Usuario pidio agregar verificacion formal de identidad con DNI y licencia de conducir	Ficha inicial no incluia verificacion de identidad (solo registro de perfil)	Aceptada con modificacion: se agrega verificacion de identidad como requisito para el rol de conductor	Revision humana directa del alcance, sin fuente externa a verificar	Alcance y supuestos del Dia 1 actualizados en el documento	Definir si la validacion es manual o automatica, y si aplica tambien al pasajero
Dia 1	Definir el mecanismo de verificacion de identidad	Usuario precisa: revision humana en principio, con posibilidad de extender a revision automatica	Quedaba abierto si la validacion era manual o automatica	Aceptada: revision humana (admin valida documentos) en esta etapa, diseno extensible a revision automatica	Revision humana directa, sin fuente externa a verificar	Supuesto de verificacion de identidad del Dia 1 cerrado en el documento	Definir si el pasajero tambien debe verificar su DNI
Dia 1	Cerrar si el pasajero tambien verifica identidad	Usuario precisa: el pasajero tambien verifica con DNI; en realidad es un dato del perfil, no del rol	La propuesta anterior dejaba la verificacion ligada solo al rol de conductor	Modificada: la verificacion pasa a ser un atributo del PerfilRutix (una sola vez), valido para conductor y pasajero	Revision humana directa, sin fuente externa a verificar	Alcance, supuestos y decision del Dia 1 actualizados; verificacion ya no esta ligada a un rol	Ninguno: alcance del Dia 1 cerrado
Dia 1	Precisar niveles de verificacion de identidad	Usuario precisa: pasajero requiere DNI validado; conductor requiere DNI + licencia de conducir validados	La propuesta anterior trataba la verificacion como un unico nivel, igual para ambos roles	Modificada: dos niveles sobre el PerfilRutix (DNI para pasajero; DNI + licencia para conductor)	Revision humana directa, sin fuente externa a verificar	Alcance, supuestos y decision del Dia 1 actualizados con los dos niveles de verificacion	Ninguno: alcance del Dia 1 cerrado
Dia 2	Explorar al menos dos enfoques para como se conectan conductor y pasajero	Usuario pidio continuar con el TP; se le propuso comparar viaje programado vs matching on-demand	Tabla comparativa: Enfoque A (viaje programado con reserva anticipada) vs Enfoque B (matching on-demand en tiempo real), con beneficios, limites, dependencias y riesgos	Aceptada: Enfoque A, por ser el ya modelado en el codigo y el que responde al caso de uso de Rutix	Contrastado contra el dominio Viaje/Reserva ya implementado en el backend (fecha/hora fija, asientos, reservas)	Dia 2 (alternativas y decision inicial) agregado al documento	Confirmar con Ezequiel si el enfoque elegido le sirve tal cual

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 3	Definir requisitos, reglas de negocio, casos de uso y criterios de aceptacion	Usuario confirmo el Enfoque A del Dia 2 y pidio seguir con el Dia 3	10 requisitos funcionales y 11 reglas de negocio extraidos de los comandos/consultas y clases ReglaNegocio existentes en el backend	Aceptada sin cambios	Contrastado linea por linea contra RegistrarViajeComando, RegistrarReservaEnViajeComando, RegistrarPagoDeReservaComando y las clases ReglaNegocio del backend	Dia 3 (especificacion funcional) agregado al documento	Revisar en el Dia 9: viaje nace en EN_CURSO (no CREADO), pago no exige confirmacion previa, y dos reglas placeholder sin implementar
Dia 3	Resolver el hallazgo: quien y cuando pasa el viaje a EN_CURSO	Usuario: el conductor lo marca manualmente al empezar a buscar pasajeros; dispara notificacion a todos los pasajeros del viaje; no se desarrolla en este TP, se releva como caso de uso	El codigo ya fue refactorizado para que el viaje nazca en CREADO (RegistrarViaje Comando), pero faltaba definir la transicion a EN_CURSO	Aceptada: se agrega como RF11 y como caso de uso 6, marcados explicitamente como no implementados en este TP	Revision humana directa, sin fuente externa a verificar	Dia 3 actualizado: RF11, caso de uso 6 y hallazgo del estado inicial cerrado	Diseñar el mecanismo de notificaciones (push, email) cuando se aborde esa funcionalidad
Dia 4	Trazar estados y flujo reales de Viaje, Reserva y Pago	Usuario confirmo avanzar al Dia 4 tras cerrar el Dia 3	Dos diagramas de estados (Reserva, Viaje) construidos a partir de los Comando reales; se detecto que ActualizarReservaDeViajeComando y ActualizarViaje Comando (PUT genericos) permiten cambiar el estado sin pasar por las reglas de negocio	Aceptada sin cambios	Contrastado contra ActualizarViaje Comando, EliminarViajeComando, ActualizarReservaDeViajeComando, ConfirmarReservaDeViajeComando, RechazarReservaDeViajeComando y CancelarReservaDeViajeComando	Dia 4 (diagramas de estados de Reserva y Viaje) agregado al documento	Revisar en el Dia 9: brecha de los PUT genericos sobre estado, y si pagar debe exigir reserva CONFIRMADA
Dia 5	Modelar datos y justificar la base elegida	Usuario confirmo avanzar al Dia 5 tras cerrar el Dia 4 y una lectura de Codigo Limpio	Diagrama entidad-relacion de las 7 entidades (PerfilRutix, Vehiculo, Tarjeta, CuentaBancaria , Viaje, Reserva, Pago) mas justificacion de MariaDB como base relacional, construidos a partir de las entidades JPA reales y application-local.yml	Aceptada sin cambios	Contrastado contra PerfilRutix, Vehiculo, Tarjeta, CuentaBancaria , Viaje, Reserva, Pago y application-local.yml (datasource, ddl-auto)	Dia 5 (modelo de datos y justificacion de la base) agregado al documento	Revisar en el Dia 9: falta de migraciones versionadas (ddl-auto=update) y fetch EAGER en colecciones de Perfil y Viaje

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 6	Mapear arquitectura e integraciones reales del ecosistema	Usuario confirmo avanzar al Dia 6 tras aplicar fetch LAZY y Flyway en el backend	Diagrama de componentes (App movil, Gateway, Rutix, MariaDB, Eureka/Config/Admin, RabbitMQ, Identidad JWT) y tabla de contrato real, construidos a partir de los controllers, el pom.xml, SecurityConfig y JwtUtils	Aceptada sin cambios	Contrastado contra los 6 controllers, pom.xml (sin FeignClient, con amqp sin uso), application-local.yml, SecurityConfig.java y JwtUtils.java	Dia 6 (arquitectura e integraciones) agregado al documento	Revisar en el Dia 9: SecurityConfig con anyRequest().permitAll() y clave RSA usada como secreto HMAC
Dia 3-4 (actualizacion)	Registrar un cambio real del backend hecho en paralelo al TP, fuera de los dias en curso	Usuario pidio (fuera del TP) que el pago de una reserva solo sea posible si la reserva esta confirmada por el conductor	Se cambio la validacion de RegistrarPagoDeReservaComando: de exigir PENDIENTE paso a exigir CONFIRMADA	Aplicada en el backend real	Contrastado contra el archivo actual de RegistrarPagoDeReservaComando	Codigo real modificado. RN9 (Dia 3) y el hallazgo de Pago (Dia 4) describen el comportamiento viejo (pago en PENDIENTE): quedan desactualizados, no se reescriben para no perder el registro de la evolucion	Revisar y actualizar RN9 y el hallazgo de Pago en la auditoria final (Dia 10)
Dia 3-4 (actualizacion)	Registrar restricciones agregadas a ActualizarReservaDeViajeComando e implementacion real de IniciarViaje	Usuario pidio (fuera del TP) restringir el PUT generico de reservas al pasajero dueno en estado PENDIENTE respetando asientos disponibles, corregir un bug de constructor faltante, e implementar el caso de uso IniciarViaje siguiendo el patron del proyecto	ActualizarReservaDeViajeComando ya no es un PUT libre: valida dueno, estado PENDIENTE y asientos disponibles. Se creo IniciarViajeComando (solo el conductor, CREADO a EN_CURSO)	Aplicada en el backend real, de forma incremental en varios intercambios	Contrastado contra ActualizarReservaDeViajeComando, Viaje.java e IniciarViajeComando actuales	Codigo real modificado. RF11 y el caso de uso 6 (Dia 3, marcados como no implementados) y el hallazgo de bypass del PUT (Dia 4) quedan desactualizados: Iniciar Viaje ya existe y el PUT de reservas ya no bypassea todo (aunque sigue seteando estado directo del DTO sin pasar por confirmar/rechazar/cancelar)	Revisar y actualizar RF11, caso de uso 6 y el hallazgo del PUT en la auditoria final (Dia 10)
Dia 5 (actualizacion)	Resolver el hallazgo de migraciones no versionadas detectado en el propio Dia 5	Usuario pregunto que se podia hacer con el tema de migraciones versionadas y pidio implementarlo	Se agrego Flyway (flyway-core, flyway-mysql), un baseline V1 reconstruido desde las entidades JPA, y ddl-auto paso de update a validate	Aplicada en el backend real	Contrastado contra pom.xml, application-local.yml y V1__baseline.sql	Codigo real modificado. El hallazgo de ddl-auto sin migraciones versionadas (Dia 5) ya no aplica: queda desactualizado	Marcar como resuelto el hallazgo de migraciones del Dia 5 en la auditoria final (Dia 10); verificar el baseline V1 contra el esquema real antes de usarlo en un entorno nuevo

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 5 (actualizacion)	Resolver el hallazgo de fetch EAGER detectado en el propio Dia 5	Usuario pidio pasar las colecciones EAGER a LAZY, con miedo de romper alguna consulta	Se analizaron los usos reales de cada coleccion (grep en todo el codigo): solo 5 lugares las leen, ninguno en los listados paginados. Se cambiaron las 4 colecciones de EAGER a LAZY	Aplicada en el backend real	Contrastado contra PerfilRutix.java, Viaje.java y los usos reales de getListaReservas/Vehiculos/Tarjetas/Cuentas	Codigo real modificado. El hallazgo de fetch EAGER (Dia 5) ya no aplica: queda desactualizado	Marcar como resuelto el hallazgo de fetch EAGER del Dia 5 en la auditoria final (Dia 10)
Dia 6 (actualizacion)	Resolver el hallazgo de permitAll() detectado en el propio Dia 6	Usuario consulto que recomendacion habia sobre seguridad, aclarando que la identidad vive en otro microservicio pero evaluaba traerla a Rutix y no depender del Gateway; se recomendo no mover la emision de tokens pero si exigir authenticated() en Rutix, y el usuario pidio implementarlo	SecurityConfig paso de permitAll() a authenticated() (con excepcion de /actuator y /swagger-ui); JwtTokenFilter ahora captura token invalido y devuelve 401 en vez de 500; se agrego CustomAuthenticationEntryPoint para devolver 401 (no el 403 vacio por defecto) cuando no hay token	Aplicada en el backend real	Contrastado contra SecurityConfig.java, JwtTokenFilter.java y CustomAuthenticationEntryPoint.java actuales	Codigo real modificado. El hallazgo de permitAll() (Dia 6) ya no aplica: queda desactualizado	Marcar como resuelto el hallazgo de permitAll() del Dia 6 en la auditoria final (Dia 10); el secreto HMAC en texto plano sigue pendiente
Dia 7	Mapear los recorridos centrales de la interfaz real de Rutix (pasajero y conductor)	Usuario confirmo avanzar al Dia 7	Diagrama de recorridos (login, tabs, busqueda/listado, publicar viaje, Detalle del viaje compartido por rol) y tabla de pantallas, construidos a partir del frontend Ionic/Angular real (rutas, paginas, componentes)	Aceptada sin cambios	Contrastado contra app.routes.ts, tabs.routes.ts, viajes.routes.ts, cuenta.routes.ts, InicioPage, ViajeDetallePage, ReservaCardComponent, ViajeFormModalComponent y LoginPage	Dia 7 (recorridos centrales) agregado al documento. Se detecto una inconsistencia nueva front/back: el boton PAGAR se muestra en PENDIENTE pero el backend ya exige CONFIRMADA	Agregados H13 y H14 al backlog de hallazgos; revisar en el Dia 9
Dia 7 (actualizacion)	Resolver el hallazgo H14 detectado en el propio Dia 7	Usuario pidio refactorizar para que el boton Pagar se muestre cuando la reserva este confirmada y reflejar el cambio en la documentacion del TP	Se separo el *ngIf de reserva-card.component.html: Cancelar sigue atado a PENDIENTE (unico estado que permite Reserva.cancelar() en el backend) y PAGAR paso a depender de CONFIRMADA, alineado con RegistrarPagoDeReservaComando	Aplicada en el frontend real	Contrastado contra Reserva.java (confirmar/cancelar) y RegistrarPagoDeReservaComando antes de editar reserva-card.component.html	Codigo real modificado. El hallazgo H14 del backlog se marca Resuelto. El hallazgo (nuevo, Alta) descrito en el propio Dia 7 queda desactualizado: ya no aplica, no se reescribe para no perder el registro de la evolucion	Revisar y actualizar el hallazgo del Dia 7 en la auditoria final (Dia 9/10)

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 3-4 (actualizacion)	Registrar una funcionalidad nueva agregada al backend/frontend reales, fuera de los dias en curso	Usuario pidio que, al confirmar una reserva, el sistema valide si los asientos confirmados (incluyendo la actual) igualan los asientos del viaje y en ese caso lo pase a un estado completado; y que el conductor tambien pueda invocar esa misma transicion manualmente desde un boton, para cerrar reservas e iniciar el viaje mas adelante aunque queden lugares disponibles	Se agrego el estado COMPLETO a EstadoViajeEnum (entre CREADO y EN_CURSO). Viaje.confirmarReserva() ahora completa automaticamente el viaje cuando los asientos confirmados igualan el total. Se creo CompletarViajeComando (solo el conductor, desde CREADO) para el disparo manual, con endpoint PUT /viajes/id/completar. IniciarViajeComando ahora acepta CREADO o COMPLETO. Se agrego la accion VIAJE_COMPLETAR en BuscarIdViajeConsulta y el boton correspondiente en ViajeDetallePage	Aceptada sin cambios	Contrastado contra Viaje.java, Reserva.java, ConfirmarReservaDeViajeComando, IniciarViajeComando y BuscarIdViajeConsulta antes de editar	Codigo real modificado (backend y frontend). El diagrama de estados del Viaje (Dia 4) y las reglas/casos de uso de reservas y viajes (Dia 3) quedan desactualizados: no incluyen el estado COMPLETO ni esta transicion automatica/manual; no se reescriben para no perder el registro de la evolucion	Agregar el estado COMPLETO y el caso de uso de completar viaje al diagrama y la especificacion del Dia 3/4 en la auditoria final (Dia 9/10); verificar compilacion y el flujo completo (reservar, confirmar hasta llenar, completar automatico/manual, iniciar)

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 3-4 (reconciliacion anticipada)	Actualizar la documentacion del Dia 3/4 con el estado COMPLETO, en vez de esperar a la auditoria final	Usuario pidio explicitamente actualizar la documentacion con la modificacion de COMPLETO, en el mismo intercambio en que se implemento	Se redibujo el diagrama de estados del Viaje (nodo 0f96ca55-3e9a): CREADO y COMPLETO en vivo, EN_CURSO en vivo (via IniciarViajeComando, con flecha directa desde CREADO y via COMPLETO), FINALIZADO y CANCELADO siguen future/discontinuos. Se reescribieron los dos parrafos de prosa bajo 'Viaje: una sola transicion real hoy' y el caso de uso 6 del Dia 3 (ya no 'no implementado'); se agrego un caso de uso 7, Completar viaje	Aceptada sin cambios	Contrastado contra el codigo real ya modificado (Viaje.java, CompletarViajeComando, IniciarViajeComando) antes de redibujar y reescribir	Documento del TP actualizado: embed y caption del Dia 4 reescritos, dos parrafos de prosa reescritos, caso de uso 6 reescrito y caso de uso 7 agregado en el Dia 3. Esto es una excepcion puntual a la convencion de diferir reconciliaciones al Dia 9/10 (fila anterior), hecha porque el usuario la pidio de forma explicita para esta funcionalidad	Ninguno: el Dia 3/4 ya refleja el estado COMPLETO. Sigue pendiente revisar en el Dia 9/10 el resto de hallazgos diferidos (RN9/Pago, PUT generico de reservas, hallazgo del Dia 7)

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 4 (ampliacion)	Agregar diagramas de secuencia del flujo del viaje, uno desde el conductor y otro desde el pasajero	Usuario pidio un diagrama de secuencia del flujo del proceso del viaje, uno desde el conductor y otro desde el pasajero	Dos diagramas de secuencia (Pasajero/Conductor - App Ionic - API Rutix), construidos a partir de los endpoints y Comando reales: conductor cubre publicar, confirmar reserva (con el auto-completar de Viaje.confirmarReserva()), completar manual (CompletarViajeComando) e iniciar (IniciarViajeComando); pasajero cubre buscar, reservar, confirmar/rechazar, pagar (RegistrarPagoDeReservaComando) y cancelar (CancelarReservaDeViajeComando). Se agregaron como subseccion nueva al final del Dia 4 (Viaje), antes de Pago	Aceptada sin cambios	Contrastado contra ViajeController, ReservaController, Viaje.java y los Comando reales de Viaje y Reserva antes de dibujar; verificado el render de ambos diagramas por captura, sin errores	Documento del TP actualizado: nueva subseccion 'Secuencia del viaje: conductor y pasajero' en el Dia 4, con los dos diagramas	Ninguno

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 8	Relevar seguridad y validacion: riesgos, permisos y criterios de aceptacion	Usuario confirmo avanzar al Dia 8	Se releva el modelo de autenticacion (JWT stateless, SecurityConfig, JwtTokenFilter, CustomAuthenticationEntryPoint) y se contrasta, Comando por Comando, el chequeo manual de dueno (no hay roles de Spring Security en todo el backend). Se detectan 5 IDOR reales (ActualizarViajeComando, EliminarViajeComando, EliminarReservaDeViajeComando, ActualizarCuentaBancariaComando, ActualizarTarjetaComando sin chequeo de dueno) y un bug de comparacion de Long con != en RegistrarPagoDeReservaComando	Aceptada sin cambios	Contrastado contra SecurityConfig, JwtTokenFilter, JwtUtils, CustomAuthenticationEntryPoint, GlobalExceptionHandler y los Comando reales de Viaje, Reserva, Tarjeta, CuentaBancaria y Vehiculo (ViajeController y ReservaController incluidos, sin ningun @PreAuthorize/hasRole en todo el proyecto)	Dia 8 (seguridad y validacion) agregado al documento. H15-H20 agregados al backlog de hallazgos tecnicos	Esta vez NO se corrigio el codigo real (a diferencia de otros dias): son brechas de autorizacion reales en el backend en uso, se dejan documentadas para que el usuario decida cuando y como corregirlas

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 8 (actualizacion)	Corregir en el backend real las brechas de autorizacion documentadas (sin corregir) en el propio Dia 8	Usuario pregunto como implementar Spring Security en Rutix (roles conductor/pasajero/admin); se le explico que el perfil no tiene un rol fijo (el mismo PerfilRutix es conductor en un viaje y pasajero en otro) y que el caso real es autorizacion por dueno de recurso, no RBAC estatico; se propuso @PreAuthorize con beans de autorizacion por recurso. Usuario pidio implementarlo ahora	Se agrego @EnableMethodSecurity en SecurityConfig. Se crearon 3 beans de autorizacion (ViajeSecurity.esConductor, ReservaSecurity.esPasajeroDueno, PerfilRecursoSecurity.esDuenoDeCuenta/esDuenoDeTarjeta), todos resolviendo el dueno real via repositorio, nunca confiando en el id recibido del cliente. Se anotaron con @PreAuthorize: ViajeController.edit/delete, ReservaController.delete, CuentaBancariaController.edit, TarjetaController.edit. Se corrigio ademas el bug de Long comparado con != en RegistrarPagoDeReservaComando (ya relevado como H20)	Aplicada en el backend real	Contrastado contra ViajeController, ReservaController, CuentaBancariaController, TarjetaController y los DTO/entidades/repositorios reales (ViajeDto, EliminarReservaDeViajeComandoDto, CuentaBancariaDto, TarjetaDto, PerfilRutixRepository) antes de escribir cada bean; cada @PreAuthorize revisado por grep tras el cambio. Se verifico que el pom.xml hereda de spring-boot-starter-parent 3.5.0 sin combine.children="override" en el maven-compiler-plugin, por lo que el flag -parameters (necesario para que el SpEL #dto.id resuelva nombres de parametros) se mantiene heredado del parent	Codigo real modificado (3 archivos nuevos, 5 modificados). H15-H20 del backlog de hallazgos tecnicos marcados Resuelto. El cierre del Dia 8 ('este Dia 8 se centra en relevarla y documentarla, no en corregir el codigo real') queda desactualizado: el codigo ya fue corregido; no se reescribe esa prosa para no perder el registro de la evolucion	No se pudo compilar ni ejecutar el backend desde este entorno: el usuario debe compilar y probar manualmente cada endpoint (duenoreal permite, no-dueno bloquea con 403) antes de confiar en el fix. Actualizar el cierre del Dia 8 y el resto de hallazgos diferidos en la auditoria final (Dia 9/10)

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 6 (precision)	Precisar el contrato de identidad del Dia 6 con el codigo real de ms-security y config-service (proyecto hermano mep-services)	Usuario comento que usa ms-security (emisor de tokens) y config-service junto con Rutix, y pidio verificar el codigo real de ambos	Se confirmo que Authentication Controller (ms-security) expone POST /v1/auth/log-in y /v1/auth/sign-up, y que JwtUtils.createToken() firma con HMAC256. Se encontraron dos precisiones no documentadas: (1) la clave HMAC esta copiada caracter por caracter en texto plano en los application-local.yml de Rutix y de ms-security, dos repos distintos, no solo 'en texto plano' como decia el Dia 6; (2) Rutix declara bootstrap.yml contra ms-config:8081 pero config-data/ no tiene ningun rutix.yml, por lo que esa integracion no entrega nada en la practica	Aceptada sin cambios	Contrastado contra AuthenticationController.java, JwtUtils.java y ms-security.yml de ms-security, y bootstrap.yml de Rutix mas el listado real de config-data/ en mep-services	Documento del TP actualizado: parrafo nuevo agregado al Dia 6 (contrato de identidad), despues del parrafo sobre el secreto compartido. Se agrego H21 (Alta) al backlog de hallazgos tecnicos	Ninguno: es solo precision/documentacion, no se toco codigo real

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 6 (infraestructura, fuera del TP)	Cerrar la brecha encontrada en el Dia 6 (precision): Rutix declaraba bootstrap.yml contra config-service pero no tenia config que recibir	Usuario aclaro que usa config local para probar en su maquina, pero en produccion/testing va a usar config-service; pidio agregar la configuracion de Rutix en config-data	Se creo config-data/rutix.yml (mep-services) con hostnames de red docker (mariadb, ms-eureka, ms-config, ms-admin, zipkin, rabbitmq) en vez de localhost, mismo datasource, mismo flyway (ddl-auto validate), misma clave JWT. Se agrego el servicio rutix a docker-compose.yml (puerto 8096, depends_on ms-config/ms-eureka/zipkin/mariadb, mismo patron que ms-api-cliente). Se corrigio ademas un bug de copy-paste encontrado de paso: management.metrics.tags.application decia ms-tokens-materiales en el application-local.yml real de Rutix, se corrigio a rutix	Aplicada en el backend/infraestructura real (mep-services y Rutix)	YAML de rutix.yml validado con python3 (yaml.safe_load). No se pudo levantar docker compose ni compilar desde este entorno	Codigo/infraestructura real modificada: config-data/rutix.yml (nuevo), docker-compose.yml (servicio rutix agregado), application-local.yml de Rutix (tag de metricas corregido). Es trabajo de infraestructura del ecosistema mep-services, fuera del alcance formal del TP (que es sobre Rutix); se registra aqui por trazabilidad, ya que surgio de la verificacion hecha en el Dia 6	El usuario debe correr docker compose config (o un up real) para validar que rutix.yml se sirve correctamente desde ms-config antes de confiar en el fix

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 8 (actualizacion 2)	Agregar bypass de rol ADMIN a todas las validaciones de dueno sobre modificacion/eliminacion	Usuario pregunto si convenia implementar Spring Security con roles (hasRole ADMIN/USER); se verifico en ms-security que el modelo de roles ya existe (RoleEnum, authorities ROLE_X en el JWT) y que Rutix ya las reconstruye en JwtTokenFilter sin plomeria adicional. Usuario pidio que todas las operaciones de modificacion y eliminacion permitan tambien a un admin de sistema, no solo al dueno	Se creo RolUtils.esAdmin(authentication) (chequea ROLE_ADMIN). En los 5 endpoints con @PreAuthorize (ViajeController.edit/delete, ReservaController.delete, CuentaBancariaController.edit, TarjetaController.edit) se cambio el SpEL a hasRole('ADMIN') or <bean>.<metodo>. En los 5 Comando con chequeo manual (Confirmar/Rechazar/Completar/Iniciar Viaje, ActualizarReservaDeViaje) se agrego !RolUtils.esAdmin(auth) && <condicion original>. De paso se encontro y corrigio un bug nuevo, igual a H20, en RechazarReservaDeViajeComando: comparaba Long con != en vez de .equals()	Aplicada en el backend real	Contrastado contra el codigo real de ms-security (RoleEnum, UserDetailServiceImpl, JwtUtils) y de Rutix (JwtTokenFilter) antes de responder la pregunta de diseno; cada patch de Comando y controller revisado por grep tras el cambio	Codigo real modificado: RolUtils.java (nuevo), 4 controllers y 5 Comando. Se agregaron H22 (bug != en Rechazar, ya Resuelto) y H23 (falta de bypass admin, ya Resuelto) al backlog. No se toco ActualizarVehiculoComando/EliminarVehiculoComando: derivan el PerfilRutix del propio id del token, no de un id externo, por lo que no hay hoy una via para que un admin opere sobre el vehiculo de otro perfil sin cambiar la forma de esos DTO	No se pudo compilar ni ejecutar el backend desde este entorno. El usuario debe probar con un usuario ROLE_ADMIN real (emitido por ms-security) contra cada endpoint antes de confiar en el bypass. Si se quiere que un admin tambien pueda operar vehiculos ajenos, hay que rediseñar esos dos Comando para que acepten un id de perfil/vehiculo explicito en vez de derivarlo del token

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 9	Red-team y reconciliacion final de los hallazgos diferidos a lo largo del TP	Usuario confirmo avanzar al Dia 9	Se reconciliaron en el texto: RN9/caso de uso Pagar (Dia 3, ya exige CONFIRMADA), el parrafo del Dia 4 sobre la decision de pago (ya tomada), los dos hallazgos del Dia 5 (migraciones y fetch EAGER, resueltos), el hallazgo de permitAll() del Dia 6 (resuelto, con nota del secreto HMAC duplicado) y el hallazgo nuevo del Dia 7 (boton PAGAR, resuelto via H14). Se agrego la seccion Dia 9 con dos subsecciones: reconciliacion de lo diferido, y hallazgos de red-team que siguen abiertos (H3/H4, los PUT genericos sin maquina de estados) mas un hallazgo nuevo, H24: los contadores de asientos del Viaje no tienen @Version ni bloqueo, riesgo de sobreventa por lost update en reservas concurrentes	Aceptada sin cambios	Contrastado contra el texto real de cada Dia reconciliado y contra Viaje.java, ActualizarReservaDeViajeComando, ConfirmarReservaDeViajeComando y RechazarReservaDeViajeComando (grep de @Version, confirmado que no existe) antes de escribir el hallazgo nuevo	Documento del TP actualizado: 6 bloques reconciliados en los Dias 3 a 7, y nueva seccion Dia 9 agregada al final del documento. H24 agregado al backlog de hallazgos tecnicos (Pendiente, no se corrigio el codigo real: es un cambio de modelo de datos)	Decidir entre @Version (optimistic locking) o una consulta atomica de decremento para H24; seguir con el Dia 10 (consolidacion y auditoria final)

Fecha / etapa	Objetivo y contexto	Prompt o intercambio	Propuesta de IA	Decision humana	Verificacion	Cambio en el proyecto	Pendiente
Dia 10	Consolidacion y auditoria final del TP	Usuario confirmo avanzar al Dia 10	Se agrego la seccion final del TP: resumen ejecutivo, estado final del backlog (14 Resuelto / 10 Pendiente de 24 hallazgos, con los 5 Alta pendientes priorizados y una recomendacion por cada uno), una reflexion sobre el proceso de co-work (que funciono: verificacion contra codigo real antes de escribir, decision humana explicita antes de cada fix; limitacion principal: no se pudo compilar ni correr el backend en ningun momento del TP) y proximos pasos	Aceptada sin cambios	Contrastado contra el backlog real (tabla completa H1-H24) para contar resueltos/pendientes y priorizar; contrastado contra el propio TP para la reflexion de proceso	Documento del TP cerrado con el Dia 10 al final. Ningun codigo real modificado en este Dia	TP completo (Dias 1 a 10). Pendiente real (fuera del TP): resolver los 5 hallazgos Alta del backlog y compilar/probar el backend